JARVIS: A PORTABLE AI ASSISTANT

A

Major Project report

Submitted to



Jawaharlal Nehru Technological University, Hyderabad

In partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

By

VARAKALA VARUN RAJ	**(22VE1A05J8)**
BODDU LAVANYA	**(23VE5A0516)**
CHINTHAKUNTLA UDAY	**(23VE5A0517)**
NAMPALLI SHILPA	**(23VE5A0520)**

Under the Guidance

of

Dr. T.V.Divya

ASSISTANT PROFESSOR



SREYAS INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(Affiliated to JNTUH, Approved by A.I.C.T.E and Accredited by NAAC, New Delhi)

Beside InduAranya, Nagole, Hyderabad-500068, Ranga Reddy Dist.

(2022-2026)

SREYAS INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Real Time Research/societal Project Report on **"JARVIS: A PORTABLE AI ASSISTANT"** submitted by **Varakala Varun Raj, Boddu Lavanya, Chinthakuntla Uday, Nampalli Shilpa** bearing Hall ticket numbers: **22VE1A05J8, 23VE5A0516, 23VE5A0517, 23VE5A0520** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **COMPUTER SCIENCE AND ENGINEERING** from Jawaharlal Nehru Technological University, Kukatpally, Hyderabad for the academic year 2025-2026 is a record of bonafide work carried out by them under our guidance and Supervision.

.

Project Coordinator

Mr.M.Sudhakar

Assistant Professor

Head of the Department

Dr. U. M. FERNANDES DIMLO

HOD & Professor

Internal Guide

Dr.T.V.Divya

Assistant Professor

External Examiner

SREYAS INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We **Varakala Varun Raj, Boddu Lavanya, Chinthakuntla Uday, Nampalli Shilpa** bearing Hallticket number: **22VE1A05J8, 23VE5A0516, 23VE5A0517, 23VE5A0520** hereby declare that the Real Time Research/societal Project titled **JARVIS: A PORTABLE AI ASSISTANT** done by us under the guidance of **Dr.T.V.Divya, Assistant Professor** which is submitted in the partial fulfillment of the requirement for the award of the B.Tech degree in **Computer Science and Engineering at Sreyas Institute of Engineering and Technology** for Jawaharlal Nehru Technological University, Hyderabad is our original work.

VARAKALA VARUN RAJ	**(22VE1A05J8)**
BODDU LAVANYA	**(23VE5A0516)**
CHINTHAKUTLA UDAY	**(23VE5A0517)**
NAMPALLI SHILPA	**(23VE5A0520)**

ACKNOWLEDGEMENT

The successful completion of any task would be incomplete without mention of the people who made it possible through their guidance and encouragement crowns all the efforts with success.

We take this opportunity to acknowledge with thanks and deep sense of gratitude to **Dr. T.V.Divya, Assistant Professor, Department of Computer Science and Engineering** for his constant encouragement and valuable guidance during the Project work.

A Special vote of Thanks to **Dr. U. M. Fernandes Dimlo, Head of the Department** and **Mr.M.Sudhakar, Project Coordinator** who has been a source of Continuous motivation and support. They had taken time and effort to guide and correct us all through the span of this work.

We owe very much to the **Department Faculty, Principal** and the **Management** who made our term at Sreyas Institute of Engineering and Technology a stepping stone for our career. We treasure every moment we had spent in college.

Last but not the least, our heartiest gratitude to our parents and friends for their continuous encouragement and blessings. Without their support this work would not have been possible.

VARAKALA VARUN RAJ **(22VE1A05J8)**

BODDU LAVANYA **(23VE5A0516)**

CHINTHAKUTLA UDAY **(23VE5A0517)**

NAMPALLI SHILPA **(23VE5A0520)**

ABSTRACT

This project presents JARVIS: A Portable AI Assistant, a lightweight voice-controlled system designed to run directly from a USB pendrive across multiple platforms including Windows. The assistant combines both offline and online capabilities, enabling users to perform everyday tasks such as file management, media control, document reading, message replies, and intelligent web-based information retrieval. JARVIS utilizes speech recognition and text-to-speech technologies to enable natural human–computer interaction through voice commands. The system continuously listens for predefined wake words while running silently in the background, ensuring hands-free operation. Its conversational AI model provides real-time responses and task execution without requiring extensive setup on host systems. The portability, platform independence, and autonomous functioning of JARVIS make it an innovative and practical solution for personalized digital assistance, inspired by the concept of Iron Man's intelligent assistant.

KEYWORDS:

Portable AI Assistant, Voice Recognition, Speech-to-Text, Text-to-Speech, Cross-Platform Application, USB-Based Computing, Conversational AI, Virtual Assistant, Natural Language Processing, Task Automation.

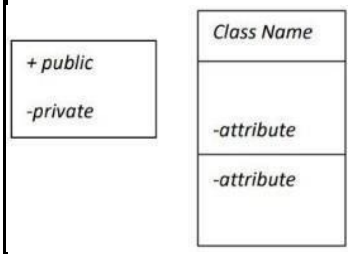
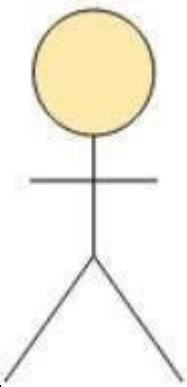
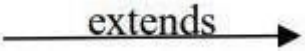
S.NO		TABLE OF CONTENTS	PAGE NO.
1		**INTRODUCTION**	**1-5**
	1.1	GENERAL	1
	1.2	EXISTING SYSTEM	2
	1.3	PROPOSED SYSTEM	3-4
	1.4.1	DISADVANTAGES OF THE EXISTING SYSTEM	5
	1.4.2	ADVANTAGES OF THE PROPOSED SYSTEM	6
2		**LITERATURE SURVEY**	**7-8**
	2.1	TECHNICAL PAPERS	7-8
3	3.1	**REQUIREMENTS**	**9-10**
	3.2	GENERAL	9
	3.3	HARDWARE REQUIREMENTS	9
	3.4	SOFTWARE REQUIREMENTS	10
4	4.1	**WEB DESIGN**	**11-20**
	4.2	GENERAL	11-12
	4.3	WEB ARCHITECTURE	13
	4.3.1	USE-CASE DIAGRAM	13-16
	4.3.2	CLASS DIAGRAM	16-18
	4.3.3	ACTIVITY DIAGRAM	18-20

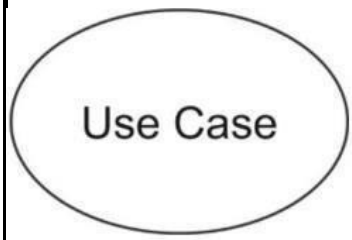
5		**TECHNOLOGY DESCRIPTION**	**21-25**
	5.1	WHAT IS HTML, CSS, JAVA SCRIPT ?	21
	5.2	ADVANTAGES OF HTML, CSS, JAVA SCRIPT	22-23
	5.3	DISADVANTAGES OF HTML, CSS, JAVA SCRIPT	24-25
6		**IMPLEMENTATION**	**26-44**
	6.1	METHODOLOGY	26-28
	6.2	SAMPLE CODE	29-44
7		**TESTING**	45-49
	7.1	GENERAL	45-48
	7.2	TYPES OF TESTING	49
	7.3	TEST CASES	49
8		**RESULTS**	**50-51**
	8.1	RESULTS SCREENSHOTS	50-51
9	9	**FUTURE SCOPE**	**52-53**
10	10	**CONCLUSION**	**54-55**
11	11	**REFERENCES**	**56-57**

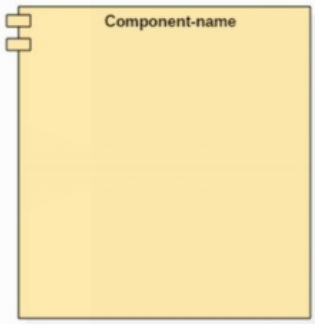
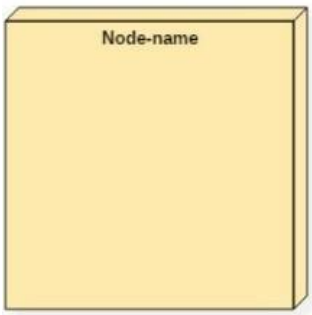
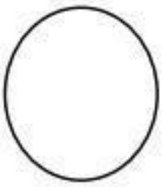
FIG. NO/TAB.NO	LIST OF FIGURES AND TABLES	PAGE NO.
4.1	Architecture Diagram	13
4.2	Use Case Diagram	16
4.3	Class Diagram	18
4.4	Activity Diagram	20
7.1	Types of Testing	49
7.2	Test Cases	49

SCREENSHOT. NO	LIST OF SCREENSHOTS	PAGE. NO
8.1	Listening Page	39
8.2	Executing Pae	40
8.3	Output Page	40

LIST OF SYMBOLS

SNO.	Name of Symbol	Notation	Description
1	CLASS		Represents a collection of similar entities grouped together.
2	ASSOCIATION		Associations represent static relationships between classes. Roles represent the way the two classes see each other.
3	ACTOR		It aggregates several classes into a single class.
4	RELATION (uses)	*Uses*	Used for additional process communication.
5	RELATION (extents)		Extends relationship is used when one use case is similar to another use case.

6	COMMUNICATION		Communication between various use cases.
7	STATE		State of the process
8	INITIAL STATE		Initial state of the object
9	FINAL STATE		Final state of the object
10	CONTROL FLOW		Represents various control flow between the states.
11	DECISION BOX		Represents decision making process from a constraint
12	USE CASE		Interact ion between the system and external environment.

13	COMPONENT		Represents physical modules which is a collection of components.
14	NODE		Represents physical modules which are a collection of components.
15	DATA PROCESS/ STATE		A circle in DFD represents a state or process which has been triggered due to some event or action.
16	EXTERNAL ENTITY		Represents external entities such as keyboard, sensors, etc
17	TRANSITION		Represents communication that occurs between processes.

18	OBJECT LIFELINE		Represents the vertical dimensions that the object communications.
19	MESSAGE	Message	Represents the message exchanged.

CHAPTER 1
INTRODUCTION

1.1GENERAL

JARVIS: A Portable AI Assistant – General Info

JARVIS: A Portable AI Assistant is an innovative voice-controlled system developed to operate directly from a USB pendrive, enabling seamless use across multiple platforms such as Windows without requiring traditional installation procedures. The assistant is designed to provide natural human–computer interaction through speech recognition and text-to-speech technologies, allowing users to communicate using simple voice commands. It continuously runs in the background, remaining inactive until the detection of a predefined wake word, after which it activates to interpret and execute user requests. Using Natural Language Processing (NLP), JARVIS understands user intent and performs tasks including file searching, document reading, application launching, media control, message handling, and intelligent information retrieval. The system supports both offline and online functionalities, ensuring reliable task execution even without internet connectivity, while enabling advanced conversational AI responses when online. All processing operations and user-specific configurations are stored within the portable device, providing strong privacy protection and independence from cloud storage. Its lightweight framework ensures fast execution with minimal system resource usage. By combining portability, cross-platform capability, autonomous command handling, and secure voice interaction, JARVIS presents a practical and accessible digital assistant solution inspired by futuristic AI systems, offering continuous personalized support for academic, professional, and general computing needs.

Key Points:

- **Portable Execution:** Operates directly from a USB pendrive without installation on Windows.
- **Voice-Based Interaction:** Enables hands-free control using speech recognition and text-to-speech technologies.
- **Offline & Online Support:** Performs local tasks offline and provides AI-based information retrieval when internet is available.
- **Secure & Autonomous Operation:** Runs quietly in the background with wake-word activation while keeping user data locally stored for privacy.

1.2 EXISTING SYSTEM

In the current scenario, virtual assistants such as **Google Assistant, Siri, Alexa, and Cortana** function mainly as cloud-based services embedded within specific devices or operating systems. These assistants depend heavily on continuous internet connectivity to process voice commands and provide responses, as most of the speech recognition and AI processing occurs on remote servers. Their functionality becomes limited or unavailable in offline or poor network conditions, reducing reliability for users who require consistent assistance.

Most existing assistants must be **installed separately on each device** and require **user authentication through online accounts**, making portability difficult. Preferences and data synchronization are managed through cloud services rather than being locally stored. As a result, users are unable to simply carry their personalized assistant and run it across different computers without performing repeated installations and setups, which leads to inconvenience and lack of seamless usage across multiple systems.

Another drawback is **platform dependency**. Many assistants are optimized for particular environments such as mobile operating systems or proprietary ecosystems. Full functionality may not be available on all platforms, especially on desktop operating systems or Linux-based systems. This limited cross-platform support restricts their usefulness for individuals who work across **Windows** devices and require a truly universal assistant solution.

From a privacy and security perspective, existing systems store user data including voice recordings, preferences, and conversation histories on cloud servers. This reliance on third-party data storage raises concerns regarding unauthorized data access and reduced control over personal information. Additionally, the lack of a fully portable execution model means users cannot operate an assistant from a simple external device like a USB pendrive, highlighting the absence of a secure, offline-capable, and universally accessible AI assistant—challenges that inspire the development of JARVIS.

1.3 PROPOSED SYSTEM

1. Architecture and Portability

The proposed Jarvis system is designed to be fully decentralized and portable, operating seamlessly from a single USB pendrive across diverse Windows and Linux environments. In contrast, commercial assistants like Alexa and Google Assistant are tethered to specific, stationary hardware ecosystems, such as smart speakers or proprietary mobile applications, lacking true plug-and-play mobility.

2. Data Privacy and Security

Jarvis prioritizes extreme user privacy by utilizing local virtual environments. Sensitive API keys and wake-word audio processing remain strictly on the local hardware, ensuring no continuous cloud recording. Alexa and Google Assistant, however, continuously stream and log voice interactions to corporate data centers, which presents ongoing data-mining and privacy concerns for the end user.

3. System-Level Integration

Unlike commercial AIs that operate within restrictive software "sandboxes," the proposed system features deep operating system integration. Jarvis acts as a sovereign extension of the user, capable of executing complex file operations, running Python scripts, and directly manipulating the host PC's hardware. Google Assistant and Alexa are largely restricted to basic web queries, media playback, and authorized smart home controls.

4. Offline Functionality

The proposed system introduces a hybrid operational model. While it utilizes the cloud for heavy reasoning, its core functions□such as localized system automation, basic timekeeping, and offline dictation□function perfectly without network access. Commercial assistants are inherently internet-dependent, rendering them essentially non-functional when Wi-Fi drops.

5. Customization and Persona

Jarvis offers an open-source, highly programmable framework where the user dictates the AI's behavior. The persona is explicitly hardcoded to mimic cinematic counterparts (e.g., the strict "Sir" protocol) and can be tailored to unique professional workflows. Alexa and Google Assistant provide a generalized, one-size-fits-all corporate persona with highly restricted customization options.

6. Intelligence Engine Strategy

While Google and Amazon optimize their assistants for quick trivia, shopping integration, and standard reminders, the proposed Jarvis system is engineered for deep reasoning. By routing complex queries through Advanced Large Language Models (like the Gemini API), Jarvis is capable of generating code, summarizing long-form documents, and maintaining complex conversational context that outpaces standard consumer smart speakers.

7. Hardware Independence

Commercial assistants force users into proprietary hardware ecosystems (e.g., Amazon Echo, Google Nest) to experience their full capabilities. Jarvis is entirely hardware-agnostic. Because it is packaged in a portable Python environment, it leverages the existing microphone, CPU, and speakers of whatever host computer it is plugged into, eliminating the need to purchase specialized smart devices.

8. Extensibility and Developer Freedom

Adding a new capability to Alexa or Google Assistant requires developing a "Skill" or "Action" that must pass through strict corporate certification processes and conform to specific API formats. Jarvis allows for instantaneous extensibility; if a user wants Jarvis to control a new piece of local software, they simply write a new Python module and drop it into the source folder.

9. Monetization and Ad-Free Experience

Commercial voice assistants are intrinsically linked to retail and advertising ecosystems, often suggesting sponsored products or attempting to drive e-commerce sales. Jarvis is a fundamentally ad-free, utility-driven tool. It operates purely on the cost of its underlying API tokens (if using paid LLM models) without any corporate upselling or behavioral tracking.

10. Contextual Memory and Retention

Google and Alexa are designed for ephemeral, transactional interactions—they answer a question and quickly drop the context to clear memory. Jarvis is designed for persistent workflow assistance. Through local database integration, it can maintain session context over long coding or study sessions, remembering previous variables or instructions the user provided earlier in the day.

1.4.1 DISADVANTAGES OF EXISTING SYSTEM

- **Heavy Internet Dependence:** Most virtual assistants require continuous internet connectivity for command processing and data retrieval, making them unusable in offline or low-network conditions.
- **No True Portability:** Existing systems cannot be executed directly from removable storage devices such as USB pendrives, preventing users from carrying their personal assistant to different computers.
- **Installation Required:** Assistants must be downloaded and installed on each individual device, which is inconvenient especially on shared or restricted systems.
- **Limited Cross-Platform Compatibility:** Many assistants work best only on specific operating systems and do not offer full or consistent features across Windows, Linux, and macOS platforms.
- **Cloud Data Storage:** User conversations and voice commands are stored on remote servers, reducing personal control over sensitive data.
- **Security Risks:** Centralized online storage increases vulnerability to data breaches, hacking, and unauthorized account access.
- **Account Login Dependency:** Access to assistants requires continuous login using personal accounts, restricting private or anonymous use.
- **Repeated Configuration:** Users must set permissions and preferences separately for each device, leading to time-consuming setup and inconsistent experiences.
- **Minimal Offline Use:** When internet access is not available, assistants offer little or no functionality, even for simple tasks.
- **High Resource Usage:** Many existing assistants consume significant system resources, affecting device performance and battery life, especially on low-end systems.
- **Lack of Customization:** Most assistants offer limited personalization options, preventing users from fully tailoring command behavior or task automation to individual needs
- **Ecosystem Lock-In:** Users become dependent on specific service providers or hardware platforms, reducing flexibility to switch systems freely.

1.4.2 ADVANTAGES OF PROPOSED SYSTEM

- **True Portability:** The assistant runs directly from a USB pendrive without installation, enabling users to carry their personalized AI assistant anywhere and use it on any compatible system.
- **Cross-Platform Support**: Works seamlessly on Windows, Linux, and macOS, ensuring consistent functionality across different operating environments.
- **Offline Functionality:** Allows essential tasks such as file handling, document reading, and application control even without internet connectivity.
- **Voice-Controlled Convenience:** Provides hands-free operation using accurate speech recognition and natural text-to-speech responses.
- **Enhanced Privacy:** Performs most processing locally and stores personal data securely on the USB device rather than cloud servers.
- **Improved Data Security:** Reduces risks of data breaches and unauthorized access by minimizing external data transmission and cloud dependency.
- **No Installation Required:** Can be executed instantly on any system without administrative permissions or complex setup procedures.
- **Dual-Mode Operation:** Combines offline task management with online AI-based information retrieval for uninterrupted availability.
- **Low Resource Usage:** Lightweight design ensures efficient operation without affecting system performance or battery life.
- **Consistent User Experience:** Maintains personalized settings across different systems without requiring repeated configurations.
- **Autonomous Background Operation:** Runs silently and activates only through wake-word detection for immediate assistance**.**
- **Accessibility Enhancement:** Supports users with physical limitations by enabling full hands-free system control.

CHAPTER 2
LITERATURE

1. Speech Recognition and Audio Processing (The "Ears" of Jarvis)

Title: Robust Speech Recognition via Large-Scale Weak Supervision **Authors:** Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (OpenAI) **Year:** 2022

Abstract: We study the capabilities of speech processing systems trained simply to predict large amounts of transcripts of audio on the internet. When scaled to 680,000 hours of multilingual and multitask supervision, the resulting models generalize well to standard benchmarks and are often competitive with prior fully supervised results but in a zero-shot transfer setting without the need for any fine-tuning. The models demonstrate high robustness to accents, background noise, and technical language.

Relevance to Project: This paper validates the use of advanced neural network-based STT (Speech-to-Text) engines (like Whisper or Google Speech API) in your project, proving that modern models can overcome the background noise and hardware limitations of portable USB microphones.

2. Large Language Models as Reasoning Engines (The "Brain" of Jarvis)

Title: The Rise and Potential of Large Language Model Based Agents: A Survey **Authors:** Xi, Z.Chen, W., Guo, X., He, W., Ding, Y., Hong, B., ... & Gui, T. **Year:** 2023

Abstract: For a long time, humanity has pursued artificial intelligence (AI) equivalent to or surpassing the human level. Recently, Large Language Models (LLMs) have demonstrated significant potential in achieving human-level intelligence. This paper provides a comprehensive survey on LLM-based agents, which transition from mere text generators to autonomous entities capable of using external tools, interacting with environments, and executing multi-step planning. The study explores how LLMs act as the cognitive brain for executing practical tasks.

Relevance to Project: This literature provides the academic justification for using the Gemini API. It proves that feeding user intent to an LLM allows Jarvis to act as an "agent" that can use tools (like your Python scripts) to control the operating system, rather than just acting as a chatbot.

3. Privacy and Portability through Edge Computing

Title: Edge Intelligence: Paving the Last Mile of Artificial Intelligence with Edge Computing **Authors:** Zhou, Z., Chen, X., Li, E., Zeng, L., Luo, K., & Zhang, J. **Year:** 2019

Abstract: As the Internet of Things (IoT) proliferates, a massive amount of data is generated at the network edge. Cloud-based AI faces challenges including high latency, lack of data security, and heavy network bandwidth consumption. This paper introduces "Edge Intelligence," exploring the deployment of AI models directly on local, resource-constrained devices. It evaluates frameworks that allow machine learning algorithms to execute locally, ensuring user privacy and real-time responsiveness.

Relevance to Project: This directly supports the core novelty of your project: **Portability**. By isolating the virtual environment on a USB drive and executing system commands locally (Edge Computing), Jarvis protects user data and avoids the latency of sending sensitive system information to the cloud.

4. Limitations of Current Commercial Virtual Assistants

Title: Next-Generation of Virtual Personal Assistants (Microsoft Cortana, Apple Siri, Amazon Alexa, and Google Home) **Authors:** Kepuska, V., & Bohouta, G. **Year:** 2018

Abstract: This paper reviews the current state of voice-activated Virtual Personal Assistants (VPAs). It compares the architecture, speech recognition accuracy, and task-execution capabilities of major commercial systems. While these systems excel in cloud-based information retrieval and smart-home integration, the study identifies critical limitations regarding cross-platform operability, deep desktop environment integration, and open-source customizability for developers.

Relevance to Project: This paper establishes the "Problem Statement" of your report. It highlights exactly what Alexa and Google Assistant *cannot* do (control local PC files, run offline, offer custom programmable personas), which perfectly sets up Jarvis as the proposed solutio

CHAPTER 3

TECHNICAL REQUIREMENTS

3.1 GENERAL

These are the technical requirements necessary for the successful development and execution of the project **"JARVIS: A Portable AI Assistant."** They specify both hardware and software resources required to support real-time voice interaction, natural language processing, task automation, and portable cross-platform operation. These requirements ensure that the system performs efficiently and reliably in both offline and online environments.

They are:

1. Hardware Requirements
2. Software Requirements

3.2 HARDWARE REQUIREMENTS

The hardware requirements serve as the foundation for system performance and reliability. They ensure that voice processing, background listening, natural language understanding, and multitasking operations occur smoothly without system delays or interruptions. These specifications also accommodate development tasks and everyday execution from portable storage devices.

- **Processor** : Minimum **Intel i5** or equivalent
- **RAM** : Minimum **8 GB**
- **Hard Disk** : Minimum **250 GB** free space
- **Graphics Card** : Minimum **4 GB** (for smooth interface and multimedia handling)
- **USB Pendrive** : Minimum **8 GB** for portable execution of JARVIS
- **Microphone & Speakers** : For voice input and audio output

3.3 SOFTWARE REQUIREMENTS

The software requirements detail the tools, platforms, and technologies required to design, develop, and deploy the JARVIS AI assistant. These components enable voice recognition, NLP processing, conversational responses, automation tasks, and seamless cross-platform execution.

- **Operating System** : Windows / Linux / macOS
- **Programming Language** : Python
- **Code Editor** : Visual Studio Code (VS Code)
- **Development Environment** : Virtual Environment for package isolation
- **Speech Recognition Libraries** : SpeechRecognition, PyAudio
- **Text-to-Speech Libraries** : pyttsx3 or equivalent APIs
- **NLP Tools** : NLTK, spaCy, or any LLM-based API
- **Automation & Integration Modules** : webbrowser, OS command libraries, Wikipedia API
- **Portable Runtime Setup** : Enables execution directly from USB without installation

CHAPTER-4

DESIGN

4.1 GENERAL

The development of Jarvis encompasses a wide range of elements and principles aimed at creating an AI assistant that is responsive, intelligent, and highly functional across multiple platforms. Here are the general matters in the Jarvis AI design:

1. Architecture & Logic

- **Modular Systems:** Use of independent modules for Vision, Voice, and Logic to ensure stability.
- **Task Hierarchy:** Prioritization of system commands vs. general conversation to ensure rapid execution.

2. Natural Language Processing (NLP)

- **Intent Recognition:** Choosing models (like Gemini or GPT) that accurately interpret user requests.
- **Context Retention:** Ensuring the AI remembers previous prompts for a fluid, human-like conversation.
- **Prompt Engineering:** Refining system instructions to ensure the AI remains helpful and concise.

3. Voice & Audio Theory

- **STT (Speech-to-Text):** Implementing low-latency engines like OpenAI Whisper for accurate transcription.
- **TTS (Text-to-Speech):** Using high-quality synthesis (e.g., ElevenLabs or Pyttsx3) for a natural-sounding persona.
- **Wake-word Detection:** Ensuring the "Jarvis" trigger is responsive but ignores background noise.

4. System Integration

- **Hardware Control:** Ability to manage OS-level tasks like volume, brightness, and file management.
- **API Ecosystem:** Connecting Jarvis to external services like Weather, News, and Google Workspace.

- **Cross-Platform Portability:** Ensuring the assistant runs seamlessly from a USB drive on both Windows and Linux.

5. User Interface (UI)

- **Minimalist Console:** A clean, non-intrusive command-line or GUI window for visual feedback.
- **Visual Indicators:** Using animations or icons to show when Jarvis is "Listening," "Thinking," or "Speaking."

6. User Experience (UX)

- **Response Latency:** Optimizing code to ensure Jarvis responds in under 2 seconds.
- **Error Handling:** Providing polite, helpful feedback when a command isn't understood or the internet is down.
- **Offline Capability:** Integrating local libraries for basic tasks when no network is available.

7. Performance & Optimization

- **Memory Management:** Reducing RAM usage so the assistant doesn't slow down the host computer.
- **Concurrency:** Using threading to allow Jarvis to "listen" while "processing" other tasks.

8. Accessibility & Interaction

- **Hands-Free Operation:** Full control of the system without the need for a keyboard or mouse.
- **Multi-Modal Input:** Supporting both voice commands and typed text for different environments.

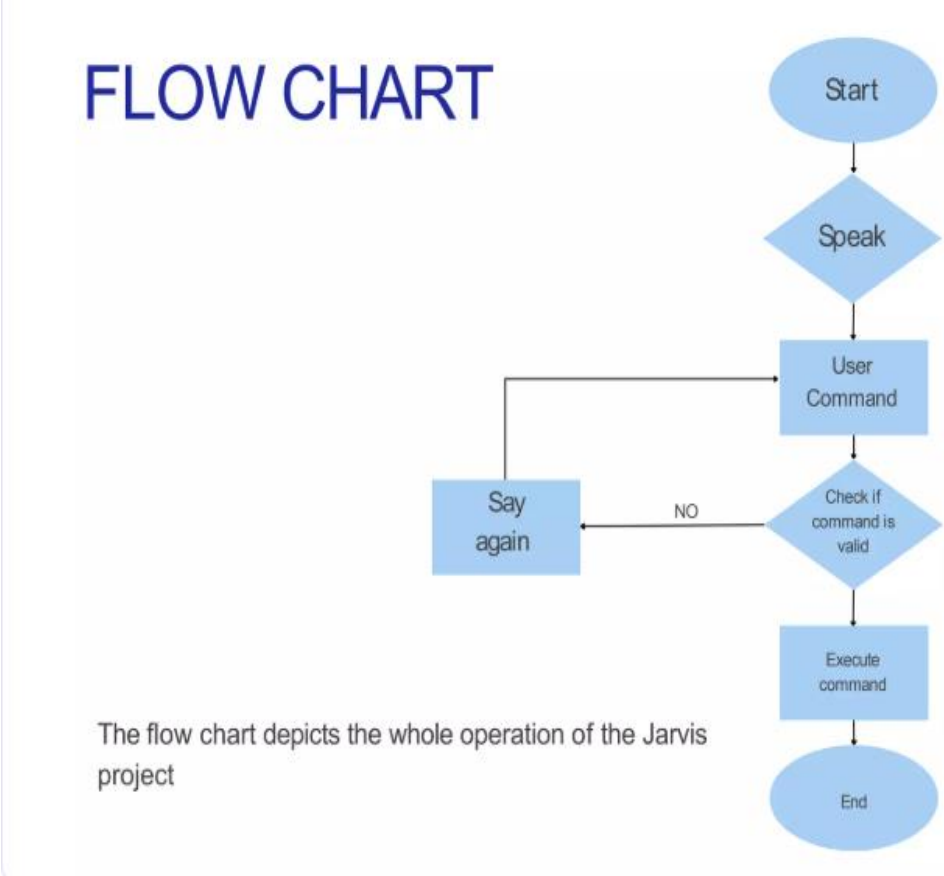
9. Security & Privacy

- **Local Processing:** Ensuring sensitive wake-word data is processed locally and not sent to the cloud.
- **API Key Encryption:** Securing user credentials and environment variables within the portable drive.

10. Maintenance & Updates

- **Modular Updates:** Designing the system so new "skills" or "capabilities" can be added without rewriting the core.
- **Log Management:** Keeping a history of interactions to debug and improve response accuracy over time.

4.2 ARCHITECTURE DIAGRAM



4.2: Architecture Diagram

4.3.1 USE-CASE DIAGRAM

A Use Case is a kind of behavioral classifier that represents a declaration of an offered behavior.

Each use case specifies some behavior, possibly including variants that the subject can perform in collaboration with one or more actors. Use cases define the offered behavior of the subject without reference to its internal structure.

These behaviors, involving interactions between the actor and the subject, may result in changes to the state of the subject and communications with its environment. A use case can include possible variations of its basic behavior, including exceptional.

The primary components of a use case diagram include:

- **Actor**

An actor is an external entity that interacts with the system. Actors can be people, other systems, or even hardware devices. Actors are represented as stick figures or simple icons.

They are placed outside the system boundary, typically on the left or top of the diagram.

- **Use Case**

A use case represents a specific functionality or action that the system can perform in response to an actor's request. Use cases are represented as ovals within the system boundary.

The name of the use case is written inside the oval.

- **Association Relationship**

An association relationship is a line connecting an actor to a use case. It represents the interaction or communication between an actor and a use case.

The arrowhead indicates the direction of the interaction, typically pointing from the actor to the use case.

1. System Architecture (The Backend Design)

This is how the internal "organs" of your code are arranged. We designed Jarvis using a **Modular architecture**, which means different parts do specific jobs without tripping over each other.

The Sensors (Input): The speech_recognition module acts as the ears, constantly listening to the microphone and filtering out background noise.

- **The Brain (Processing):** The code routes commands like a traffic cop. Simple requests (like opening Notepad) are handled instantly by the local PC. Complex requests are securely packaged and sent to the OpenAI neural network.
- **The Actuators (Output):** The pyttsx3 engine acts as the vocal cords, while pyautogui and webbrowser act as the hands to physically manipulate your computer.
- **Asynchronous Design:** We used threading so that the graphical interface doesn't freeze or lock up while Jarvis is listening or thinking.

2. User Interface / UI (The Visual Design)

This is the physical window you see on your screen. Currently, we have designed a **Minimalist Cyberpunk HUD** (Heads-Up Display).

- **Color Palette:** We used a deep black background (#050505) with high-contrast, glowing accents (Neon Green for online status, Cyan for executing commands).
- **Typography:** We utilized stark, technical fonts (Impact for major status alerts, Consolas for the rolling logs) to make it feel like a military or sci-fi terminal.
- **Data Visualization:** The bottom of the window features a live telemetry feed showing real-time CPU usage and Battery life, reinforcing the idea that Jarvis is monitoring the system.

3. User Experience / UX (The Interaction Design)

This is how it *feels* to talk to Jarvis. Good UX makes the software feel alive rather than like a rigid script.

- **Feedback Loops:** We designed Jarvis to always acknowledge a command *before* executing it (e.g., he says "Searching the web" *before* Chrome opens). This prevents the user from wondering if he heard them.
- **Contextual Memory:** By passing the last 10 lines of conversation back to ChatGPT, the UX mimics a real human conversation where you don't have to repeat the subject of the sentence every time.
- **Fuzzy Logic:** We designed the command triggers to be flexible. Instead of demanding you say exactly "search for [x]", the system catches variations like "look up" or "find", making the interaction feel natural.

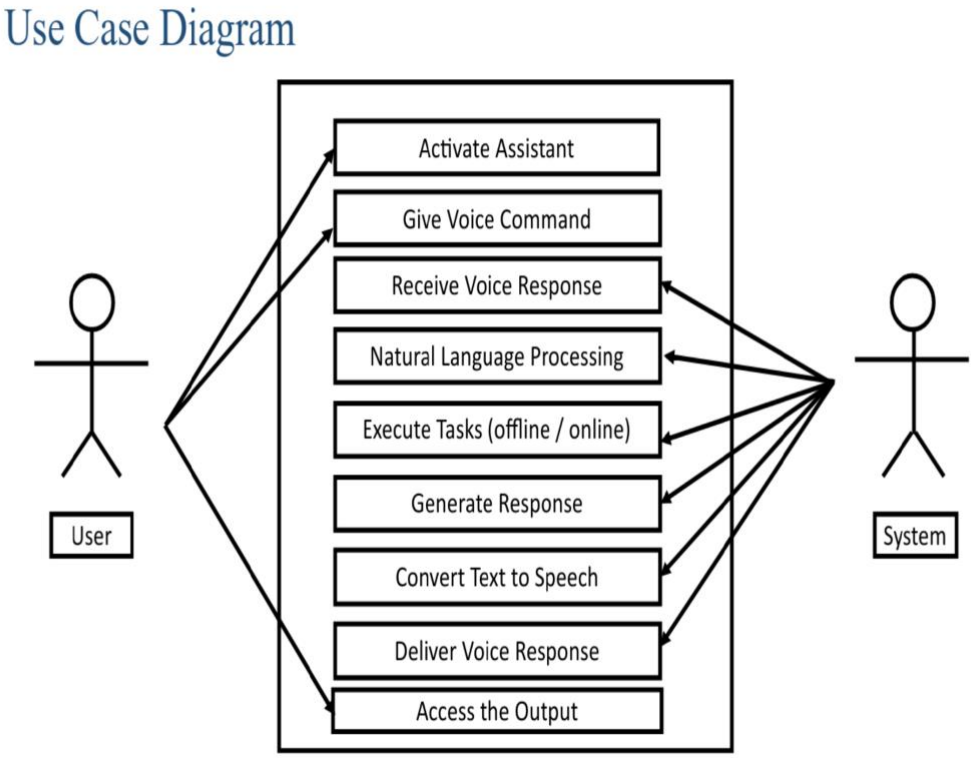


Figure 4.2: Use-Case-Diagram

4.3.2 CLASS DIAGRAM

A class diagram in Unified Modeling Language (UML) is a type of structural diagram that represents the static structure of a system by depicting the classes, their attributes, methods, and the relationships between them. Class diagrams are fundamental in object-oriented design and provide a blueprint for the software's architecture.

Here are the key components and notations used in a class diagram:

- **Class**

  A class represents a blueprint for creating objects. It defines the properties (attributes) and behaviors (methods) of objects belonging to that class.Classes are depicted as rectangles with three compartments: the top compartment contains the class name, the middle compartment lists the class attributes, and the bottom compartment lists the class methods.

- **Attributes**

Attributes are the data members or properties of a class, representing the state of objects. Attributes are shown in the middle compartment of the class rectangle and are typically listed as a name followed by a colon and the data type (e.g., name: String).

- **Methods**

Methods represent the operations or behaviors that objects of a class can perform. Methods are listed in the bottom compartment of the class rectangle and include the methodname, parameters, and the return type (e.g., calculateCost(parameters): ReturnType).

- **Visibility Notations**

Visibility notations indicate the access level of attributes and methods. The common notations are:

+ (public): Accessible from anywhere.

- (private): Accessible only within the class.

(protected): Accessible within the class and its subclasses.

~ (package or default): Accessible within the package.

- **Associations**

Associations represent relationships between classes, showing how they are connected. Associations are typically represented as a solid line connecting two classes. They may have multiplicity notations at both ends to indicate how many objects of each class can participate in the relationship (e.g., 1..*).

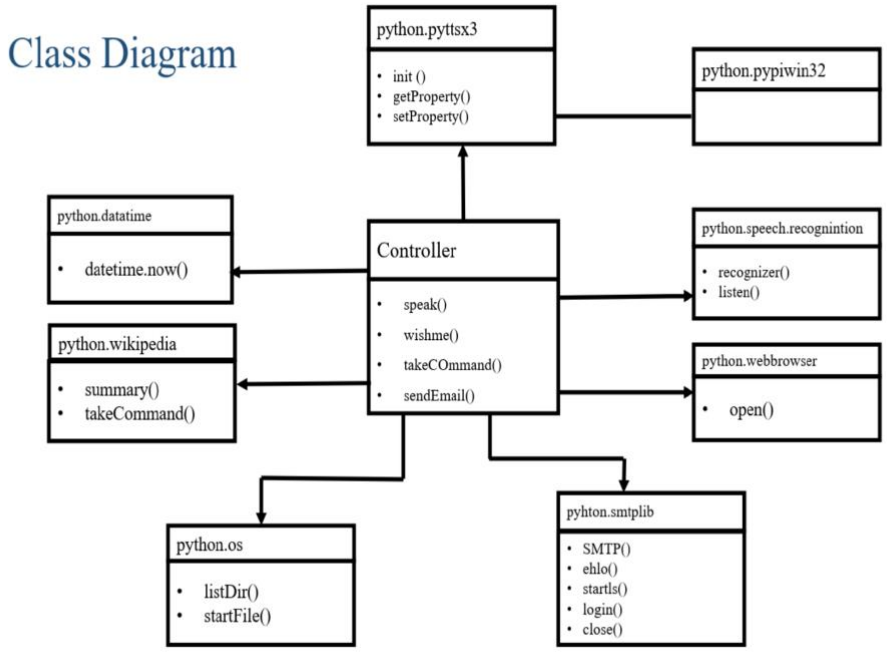


Figure 4.3: Class diagram

4.3.3 ACTIVITY DIAGRAM

An activity diagram portrays the control flow from a start point to a finish point showing the various decision paths that exist while the activity is being executed.

The diagram might start with an initial activity such as "User approaches the door." This activity triggers the system to detect the presence of the user's Bluetooth-enabled device, initiating the authentication process.

Next, the diagram could depict a decision point where the system determines whether the detected device is authorized. If the device is recognized as authorized, the diagram would proceed to the activity "Unlock the door." Conversely, if the device is not authorized, the diagram might show alternative paths such as prompting the user for additional authentication credentials or denying access.

The key components and notations used in an activity diagram:

- **Initial Node**

An initial node, represented as a solid black circle, indicates the starting point of the activity diagram. It marks where the process or activity begins.

- **Activity/Action**

An activity or action represents a specific task or operation that takes place within the system or a process. Activities are shown as rectangles with rounded corners. The name of the activity is placed inside the rectangle.

- **Control Flow Arrow**

Control flow arrows, represented as solid arrows, show the flow of control from one activity to another. They indicate the order in which activities are executed.

- **Decision Node**

A decision node is represented as a diamond shape and is used to model a decision point or branching in the process. It has multiple outgoing control flow arrows, each labeled with a condition or guard, representing the possible paths the process can take based on condition.

- **Merge Node**

A merge node, also represented as a diamond shape, is used to show the merging of multiple control flows back into a single flow.

- **Fork Node**

A fork node, represented as a black bar, is used to model the parallel execution of multiple activities or branches. It represents a point where control flow splits into multiple concurrent paths**.**

- **Join Node**

A join node, represented as a black bar, is used to show the convergence of multiple control flows, indicating that multiple paths are coming together into a single flow.

- **Final Node**

A final node, represented as a solid circle with a border, indicates the end point of the activity diagram. It marks where the process or activity concludes.

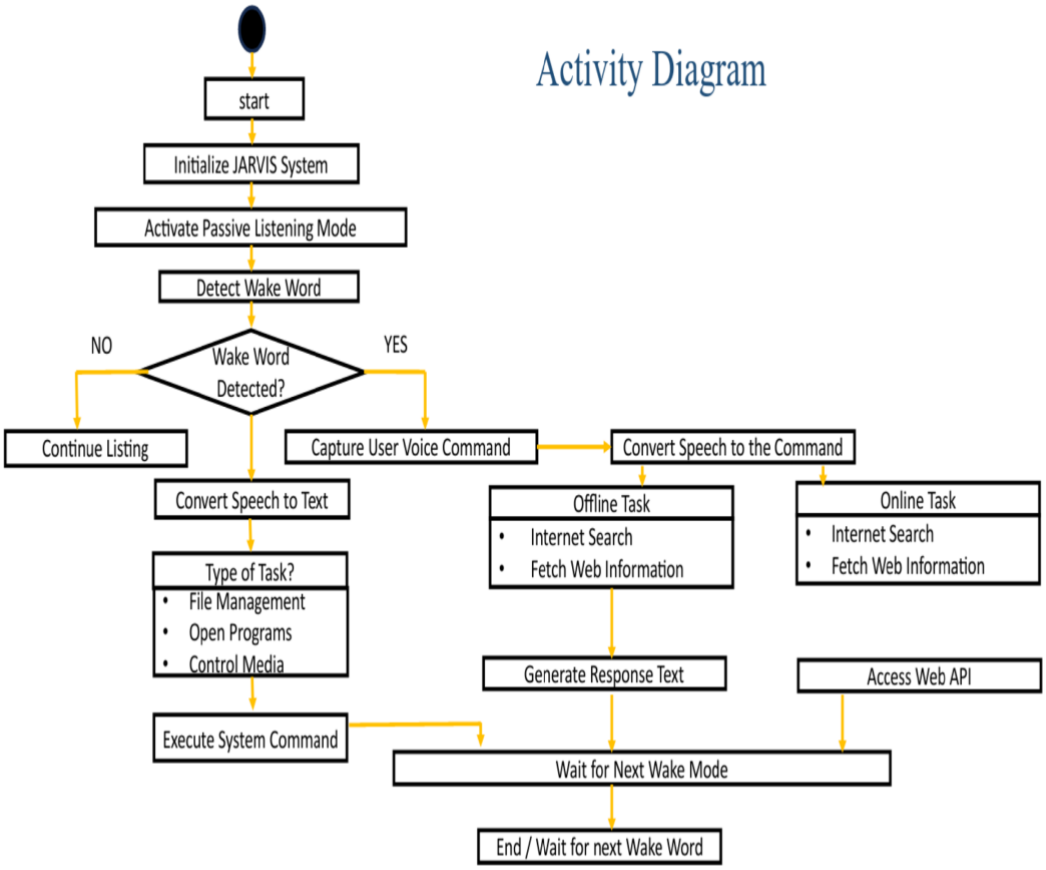


Figure 4.4: Activity diagram

CHAPTER-5
TECHNOLOGY DESCRIPTION

5.1 WHAT IS PYTHON, LLM, AND SPEECH ENGINES?

Python (The Backbone)

Python is the primary programming language used to build the structure of Jarvis. It acts as the "glue" that connects the AI's brain to the computer's hardware.

- **Modular Scripting:** Python allows Jarvis to be built in modules (e.g., main.py, voice_engine.py, actions.py).
- **Library Ecosystem:** Access to powerful libraries like PyAudio for sound, Subprocess for system control, and Requests for internet data.
- **Portability:** Python scripts can be packaged to run from a USB drive, making Jarvis truly portable.

Large Language Models / LLMs (The Brain)

The LLM (like Gemini or GPT) is the cognitive engine that allows Jarvis to understand and generate human-like text.

- **Natural Language Understanding (NLU):** It interprets the *intent* behind a user's voice command rather than just looking for keywords.
- **Contextual Memory:** It allows Jarvis to remember that "it" refers to the "weather" mentioned in a previous sentence.
- **Reasoning:** The LLM can solve problems, write code, or summarize long documents for the user.

Speech Engines: STT & TTS (The Senses)

These engines allow Jarvis to hear (Speech-to-Text) and speak (Text-to-Speech).

- **STT (OpenAI Whisper / Google):** Converts the audio waves from your microphone into text strings that the AI can read.
- **TTS (ElevenLabs / Pyttsx3):** Converts the AI's text response into a synthesized voice.
- **Wake-Word Detection:** A lightweight "always-on" listener that triggers the full system only when it hears "Jarvis."

5.2 ADVANTAGES OF THE JARVIS STACK

Python Advantages

- **Rapid Development:** Writing and debugging code is much faster than in C++ or Java.
- **System Integration:** Easily controls files, applications, and system settings.
- **Automation:** Perfect for creating "skills" like sending emails or opening browser tabs.

LLM Advantages

- **Human-Like Interaction:** Conversations feel natural rather than robotic.
- **Versatility:** One model can handle translation, coding, and general knowledge.
- **Adaptability:** The AI learns to understand different ways of phrasing the same command.

Speech Engine Advantages

- **Hands-Free Productivity:** Allows the user to multitask without needing a keyboard.
- **Accessibility:** Makes computing easier for those with visual or motor impairments.

5.3 ARCHITECTURE AND WORKFLOW OF JARVIS

The efficiency of Jarvis as a portable assistant relies on a seamless "Listen-Think-Act" loop. This architecture ensures that the assistant remains responsive while handling complex computations through a hybrid of local and cloud processing.

1. The Processing Pipeline

The workflow of Jarvis follows a linear path from the moment the user speaks until the task is completed:

- **Trigger Stage:** The **Wake-word Engine** (running locally) continuously monitors the audio stream for the keyword "Jarvis."
- **Perception Stage:** Once triggered, the **STT (Speech-to-Text)** module captures the user's command and converts the audio data into a text string.

- **Cognition Stage:** The text is sent to the **LLM (Large Language Model)**. Here, the AI analyzes the "Intent" (e.g., Is the user asking a question? Do they want to open an app?).
- **Execution Stage:** If the intent is an action, Python scripts interact with the **Operating System** (opening files, searching the web, or controlling hardware).
- **Feedback Stage:** The result is formatted into a natural response and sent to the **TTS (Text-to-Speech)** engine to be read aloud to the user.

2. Portability & Environment Setup

Since this project is designed to be portable (running from a USB drive), the architecture includes a "Virtual Environment" strategy:

- **Isolated Environment:** Uses a portable Python distribution so that Jarvis can run on any guest computer without needing to install software on the host.
- **Resource Management:** The system is optimized to use minimal RAM, ensuring it doesn't interfere with other applications running on the host machine.

3. Logic Flow (Decision Making)

Jarvis uses a conditional logic flow to determine how to handle requests:

1. **Local Skills:** If the command is "Open Notepad," the logic stays local for maximum speed.
2. **Cloud Intelligence:** If the command is "Summarize this research paper," the logic connects to the cloud LLM for high-power processing.
3. **Fallback Mechanism:** If the internet is disconnected, Jarvis switches to a basic "Offline Mode" to handle essential local system commands.

4. Data Storage & Privacy

- **Session Memory:** Short-term context is stored in a local JSON or SQLite database to maintain the "flow" of conversation.
- **Secure API Handling:** Encryption is used to store API keys on the portable drive, preventing unauthorized access if the drive is lost.

5.4 DISADVANTAGES OF THE JARVIS STACK

Python Disadvantages

- **Execution Speed:** Slower than compiled languages for heavy mathematical processing.
- **Dependency Management:** Requires ensuring all libraries are correctly installed on the portable drive.

LLM Disadvantages

- **Latency:** API-based models (Cloud) can have a delay depending on internet speed.
- **Hallucination:** The AI might occasionally provide confident but incorrect information.
- **Cost:** High-end AI models often require paid API tokens for heavy usage.

Speech Engine Disadvantages

- **Background Noise:** Low-quality microphones or loud environments can lead to incorrect transcriptions.
- **Privacy Concerns:** Always-listening wake-word engines require careful data handling to ensure user trust.

5.5 Core Technology

- **Core Programming Language (Python 3.10+):** Python serves as the backbone of the entire project. Its lightweight nature and extensive library ecosystem make it the ideal choice for creating a portable, modular application that can interact directly with Windows and Linux operating systems via the os and subprocess modules.
- **Cognitive Engine (Large Language Models):** The "brain" of Jarvis is powered by advanced LLM APIs (such as the **Google Gemini API**). Instead of relying on rigid, pre-programmed responses, the LLM processes user transcripts to understand complex intent, generate human-like conversation, and summarize web data dynamically.
- **Speech-to-Text / STT (SpeechRecognition & Whisper):** To capture user commands, the system utilizes Python's SpeechRecognition library to interface with the host computer's microphone. It filters ambient noise and converts spoken words into text strings using high-accuracy neural network models.
- **Text-to-Speech / TTS (Pyttsx3):** For the assistant's voice output, the pyttsx3 library is implemented. Unlike cloud-based voice generators, this library functions completely offline, ensuring that Jarvis can still communicate system alerts or execute basic tasks even when the internet is disconnected.

- **Web & Social Automation (Selenium & PyWhatKit):** To execute tasks outside of the local computer, Jarvis uses browser automation tools. PyWhatKit allows the assistant to autonomously send WhatsApp messages or play specific YouTube videos, while Selenium can be used to navigate complex web pages hands-free.
- **Portability Framework (Virtual Environments):** To ensure the AI is truly plug-and-play from a USB pendrive, the project utilizes isolated Python virtual environments (venv). This packages all necessary dependencies and libraries together, preventing compatibility issues with the host machine's existing software.

CHAPTER 6
IMPLEMENTATION

6.1METHODOLOGY

The methodology for the **Jarvis Portable AI Assistant** focuses on a structured approach to designing, developing, and deploying a voice-activated system that is both intelligent and mobile. This involves a systematic study of software portability, natural language

processing, and system automation to ensure Jarvis remains an efficient and secure companion. Key areas of focus include:

1. Intelligence and Logic Integration:

- **LLM Standards:** Defining and adhering to integration protocols for **Gemini Pro** and other Large Language Models to ensure high-quality reasoning.
- **Prompt Engineering:** Creating comprehensive and accurate system prompts to define Jarvis's personality (e.g., the "Sir" protocol) and operational constraints.
- **Function Mapping:** Writing and auditing Python scripts to automate local tasks like file management, web searching, and media control.

2. Portability and Environments:

- **Platform Evaluation:** Assessing the performance of Jarvis across different operating systems (Windows and Linux) to ensure core features remain stable.
- **Interoperability:** Ensuring the assistant and its virtual environment can be moved across different hardware via a **USB Pendrive** without losing user data or configuration.

3. Ethical and Data Considerations:

- **User Privacy:** Understanding and managing data rights, ensuring that voice recordings are processed locally or via secure API tunnels without being stored permanently.
- **Compliance:** Keeping abreast of AI ethics and regulations concerning automated scripts and digital assistant interactions.

4. Performance and Analytics:

- **Latency Models:** Developing methodologies to appraise the response speed of Jarvis, considering factors like internet bandwidth and local hardware processing power.
- **Resource Analytics:** Using data monitoring to track memory usage and CPU load over time to prevent system lag during long sessions.

5. Security and Authentication:

- **Voice Verification:** Implementing processes to recognize the specific frequency and tone of the user's voice to prevent unauthorized activation.
- **Encryption Measures:** Protecting API keys and personal files on the portable drive through advanced environment variable masking and local encryption.

6. Interaction and Ecosystem:

- **User Engagement:** Building a seamless conversational loop that fosters a sense of "living" technology through adaptive responses.
- **Collaborative Automation:** Working with third-party APIs (WhatsApp, YouTube, Google Search) to drive the assistant's utility across a wide digital ecosystem.

7. Sustainability and Impact:

- **Energy Efficiency:** Addressing the power consumption of local AI models by optimizing code to be "lightweight" for laptop and portable device batteries.
- **Accessibility Impact:** Considering the social implications of Jarvis by providing an eyes-free, hands-free computing experience for users with diverse physical needs.

6.2 CORE MODULES IMPLEMENTATION

The system is divided into functional modules to ensure the code remains maintainable and efficient.

1. The Auditory Module (Speech-to-Text)

This module acts as the "ears" of Jarvis. It was implemented using the SpeechRecognition library.

- **Microphone Initialization:** The script captures audio input using sr.Microphone().
- **Ambient Noise Calibration:** Before listening for commands, adjust_for_ambient_noise() is called to prevent static or background hums from causing false triggers.

- **Transcription:** The captured audio is sent through the recognize_google() method (or an offline equivalent) to convert the speech into a string variable.

2. The Vocal Module (Text-to-Speech)

To maintain the cinematic persona, the vocal output must work entirely offline. This was implemented using pyttsx3.

- **Engine Initialization:** The TTS engine is initialized, and the voice properties are adjusted to select a formal, male voice to emulate the classic "Jarvis" tone.
- **The "Speak" Function:** A central speak(audio) function was defined. Whenever the system needs to reply, it passes a string to this function, which triggers the engine.say() and engine.runAndWait() commands.

3. The Cognitive Module (LLM Integration)

For complex reasoning, Jarvis interfaces with the **Google Gemini API**.

- **API Security:** The API key is stored locally in a hidden .env file on the USB drive, accessed via the python-dotenv library to prevent hardcoding sensitive credentials.
- **Prompt Engineering:** The google-generativeai library is initialized with a specific "System Prompt" instructing the AI: *"You are Jarvis, a highly capable virtual assistant. Always address the user as 'Sir'. Keep responses concise and actionable."*
- **Generation:** When a user asks a complex question, the string is passed to model.generate_content(), and the resulting text is cleaned and sent back to the Vocal Module.

4 SYSTEM AUTOMATION IMPLEMENTATION

The execution of local tasks is handled through Python's built-in standard libraries.

- **OS Interfacing:** The os module was heavily utilized to open system directories, manage files, and launch applications without needing the mouse. (e.g., os.startfile("notepad.exe")).
- **Web Automation:** The webbrowser library was implemented to open specific URLs (like YouTube or StackOverflow), while pywhatkit was used to automate the sending of WhatsApp messages based on voice parameters.

5 THE MAIN EXECUTION LOOP

The heartbeat of Jarvis is a continuous while True loop that ties all the modules together.

1. **Wake-Word Detection:** The loop actively listens for the keyword "Jarvis".
2. **Command Parsing:** Once triggered, the assistant listens for the subsequent command and converts it to lowercase text.
3. **Logic Routing (If/Elif Chain):** * If the string contains "open google", it executes the web automation module.
    - If the string contains "what time is it", it fetches the local system time.
    - If the string contains a complex query (e.g., "explain quantum mechanics"), it routes the query to the Gemini API.
4. **Return to Sleep:** After executing the command and providing verbal feedback, the loop resets, waiting for the next wake-word.

6.3 SAMPLE CODE

```
import os
import datetime
import webbrowser
import speech_recognition as sr
import pyttsx3
import google.generativeai as genai
import threading
```

```
import tkinter as tk
import time

os.environ['PYWHATKIT_NO_INTERNET_CHECK'] = 'True'
os.environ['http_proxy'] = ""
os.environ['https_proxy'] = ""

try:
    with open("key.txt", "r") as f:
        API_KEY = f.read().strip()
except:
    API_KEY = "YOUR_API_KEY_HERE"

def connect_brain():
    genai.configure(api_key=API_KEY)
    try:
        for m in genai.list_models():
            if 'generateContent' in m.supported_generation_methods:
                return genai.GenerativeModel(m.name), m.name
    except:
        pass
    return None, "Offline"

model, model_name = connect_brain()

APP_MAP = {
    "notepad": "notepad.exe",
    "calculator": "calc.exe",
    "chrome": "chrome.exe",
    "cmd": "cmd.exe",
    "explorer": "explorer.exe",
    "settings": "start ms-settings:"
}
```

```
root = tk.Tk()
root.title(f"Jarvis ({model_name})")
root.geometry("400x150")
root.attributes("-topmost", True)
root.configure(bg="#111")
status_label = tk.Label(root, text="ONLINE", font=("Impact", 16), fg="#0f0", bg="#111")
status_label.pack(pady=10)
log_label = tk.Label(root, text="Waiting...", font=("Consolas", 10), fg="#888", bg="#111")
log_label.pack()

def update_ui(status, log):
    status_label.config(text=status)
    log_label.config(text=log)
    root.update()

def speak(text):
    try:
        engine = pyttsx3.init('sapi5')
        engine.setProperty('rate', 170)
        engine.say(text)
        engine.runAndWait()
        engine.stop()
    except:
        pass

def run_jarvis():
    r = sr.Recognizer()
    r.dynamic_energy_threshold = True
    speak("System ready.")

    while True:
        try:
```

```
with sr.Microphone() as source:
    update_ui("LISTENING...", "Ready")
    r.adjust_for_ambient_noise(source, duration=0.5)
    audio = r.listen(source, timeout=None, phrase_time_limit=10)

    update_ui("THINKING...", "Processing...")
    try:
        raw = r.recognize_google(audio).lower()
    except:
        continue

    cmd = raw.replace("jarvis", "").strip()
    update_ui("EXECUTING", f"'{cmd}'")

    if "open" in cmd and "google" not in cmd and "youtube" not in cmd:
        app = cmd.replace("open", "").replace("launch", "").strip()
        if app in APP_MAP:
            speak(f"Opening {app}")
            os.system(f"start {APP_MAP[app]}")
            continue

    elif "time" in cmd:
        now = datetime.datetime.now().strftime("%I:%M %p")
        speak(f"It is {now}")
        continue

    elif "search" in cmd or "find" in cmd or "look up" in cmd:
        topic = cmd.replace("search for", "").replace("search", "")\
                .replace("find", "").replace("look up", "")\
                .replace("in google", "").replace("on google", "").strip()
        if topic:
            speak(f"Searching Google for {topic}")
            webbrowser.open(f"https://www.google.com/search?q={topic}")
        else:
```

```
                speak("What would you like me to search for?")
            continue

        elif "open google" in cmd:
            speak("Opening Google")
            webbrowser.open("https://www.google.com")
            continue

        elif "open youtube" in cmd:
            speak("Opening YouTube")
            webbrowser.open("https://www.youtube.com")
            continue

        elif "exit" in cmd:
            speak("Goodbye.")
            os._exit(0)

        else:
            if model:
                try:
                    res = model.generate_content(f"Be concise. User: {cmd}")
                    speak(res.text)
                except:
                    speak("My AI connection is currently limited.")
            else:
                speak("I am offline.")

    except Exception:
        continue

threading.Thread(target=run_jarvis, daemon=True).start()
root.mainloop()
```

CODE SNIPPET

```
@echo off
```

```
title Jarvis Launcher
color 0a
cls
echo ----------------------------------------
echo    INITIALIZING JARVIS SYSTEMS...
echo ----------------------------------------
echo.

:: This command tells the script to look inside the 'windows' folder
:: regardless of what drive letter (D:, E:, F:) the USB uses.
cd /d "%~dp0windows"

:: Launch the executable
start jarvis.exe

:: Close this black window immediately
exit
```

KEY.TXT

```
# --- LOAD KEY FROM EXTERNAL FILE ---
try:
    # Looks for key.txt in the same folder as the .exe
    with open("key.txt", "r") as f:
        API_KEY = f.read().strip()
except:
    API_KEY = "YOUR_BACKUP_HARDCODED_KEY_HERE"
```

REQUIREMENTS.TXT

```
SpeechRecognition
pyttsx3
google-generativeai
pyaudio
```

WITH CHATGPT API:

```
import os
import datetime
import webbrowser
import speech_recognition as sr
import pyttsx3
from openai import OpenAI
import threading
import tkinter as tk
import time
import psutil
import pyautogui
import requests

# ==========================================
# 1. SYSTEM CONFIGURATION & API SETUP
# ==========================================
os.environ['PYWHATKIT_NO_INTERNET_CHECK'] = 'True'

# Securely load the OpenAI API Key
try:
    with open("key.txt", "r") as f:
        API_KEY = f.read().strip()
except:
    API_KEY = "YOUR_OPENAI_API_KEY_HERE"

# Initialize the Brain and the Memory Array
try:
    client = OpenAI(api_key=API_KEY)
    ai_status = "Connected (GPT-3.5)"
    # This array acts as Jarvis's short-term memory
```

```
    chat_history = [
        {"role": "system", "content": "You are Jarvis, a highly efficient AI assistant. Keep answers concise."}
    ]
except Exception as e:
    client = None
    ai_status = "Offline"
    chat_history = []

# ==========================================
# 2. LOCAL APPLICATION DICTIONARY
# ==========================================
APP_MAP = {
    "notepad": "notepad.exe",
    "calculator": "calc.exe",
    "chrome": "chrome.exe",
    "cmd": "cmd.exe",
    "explorer": "explorer.exe",
    "settings": "start ms-settings:",
    "task manager": "taskmgr.exe"
}

# ==========================================
# 3. GRAPHICAL USER INTERFACE (GUI)
# ==========================================
root = tk.Tk()
root.title(f"Jarvis Ultimate ({ai_status})")
root.geometry("450x200")
root.attributes("-topmost", True)
root.configure(bg="#050505")

# Main Status Text
status_label = tk.Label(root, text="SYSTEM BOOTING", font=("Impact", 18), fg="#00ffcc", bg="#050505")
```

```
status_label.pack(pady=10)

# Activity Log
log_label = tk.Label(root, text="Initializing subroutines...", font=("Consolas", 10), fg="#aaaaaa", bg="#050505")
log_label.pack()

# Hardware Stats Bar (Bottom)
stats_label = tk.Label(root, text="CPU: --% | BAT: --%", font=("Consolas", 9), fg="#555555", bg="#050505")
stats_label.pack(side="bottom", pady=10)

def update_ui(status, log):
    status_label.config(text=status)
    log_label.config(text=log)
    root.update()

def update_hardware_stats():
    """Background loop to check PC health every 3 seconds"""
    while True:
        try:
            cpu = psutil.cpu_percent(interval=None)
            battery = psutil.sensors_battery()
            bat_txt = f"{battery.percent}%" if battery else "AC Power"
            stats_label.config(text=f"CPU: {cpu}% | BAT: {bat_txt}")
        except:
            pass
        time.sleep(3)

# Start hardware monitor in the background
threading.Thread(target=update_hardware_stats, daemon=True).start()

# ==========================================
# 4. VOCAL OUTPUT ENGINE
```

```
# ==========================================
def speak(text):
    """Handles text-to-speech without locking the system"""
    print(f"Jarvis: {text}")
    try:
        engine = pyttsx3.init('sapi5')
        engine.setProperty('rate', 175) # Slightly faster, more robotic
        # Change voice to male if available
        voices = engine.getProperty('voices')
        if len(voices) > 0:
            engine.setProperty('voice', voices[0].id)

        engine.say(text)
        engine.runAndWait()
        engine.stop()
    except Exception as e:
        print(f"Audio Error: {e}")

# ==========================================
# 5. EXTERNAL MODULES (Weather & Net)
# ==========================================
def get_weather():
    """Pulls live weather data without needing an API key"""
    try:
        res = requests.get("https://wttr.in/?format=%C+%t", timeout=3)
        return res.text.strip()
    except:
        return "I cannot reach the weather satellite at this time."

# ==========================================
# 6. MAIN INTELLIGENCE LOOP
# ==========================================
def run_jarvis():
    r = sr.Recognizer()
```

```
r.dynamic_energy_threshold = True
speak("All systems are operational. I am listening.")

while True:
    try:
        with sr.Microphone() as source:
            update_ui("LISTENING...", "Awaiting command")
            r.adjust_for_ambient_noise(source, duration=0.5)
            audio = r.listen(source, timeout=None, phrase_time_limit=10)

            update_ui("PROCESSING...", "Analyzing audio")
            try:
                raw = r.recognize_google(audio).lower()
            except:
                continue # Ignore silence

            # Clean the command
            cmd = raw.replace("jarvis", "").strip()
            if not cmd:
                continue

            update_ui("EXECUTING", f"> {cmd}")

            # --- 6A. SYSTEM OVERRIDES (Instant Execution) ---
            if "exit" in cmd or "shut down" in cmd:
                speak("Powering down. Goodbye, Sir.")
                os._exit(0)

            elif "time" in cmd:
                now = datetime.datetime.now().strftime("%I:%M %p")
                speak(f"The current time is {now}")
                continue

            elif "weather" in cmd:
```

```
    speak("Checking local sensors.")
    conditions = get_weather()
    speak(f"It is currently {conditions}")
    continue

# --- 6B. HARDWARE CONTROL (PyAutoGUI) ---
elif "volume up" in cmd:
    pyautogui.press("volumeup", presses=5)
    speak("Volume increased.")
    continue

elif "volume down" in cmd:
    pyautogui.press("volumedown", presses=5)
    speak("Volume decreased.")
    continue

elif "pause" in cmd or "play music" in cmd:
    pyautogui.press("playpause")
    speak("Media toggled.")
    continue

# --- 6C. APPLICATION LAUNCHER ---
elif "open" in cmd and "google" not in cmd and "youtube" not in cmd:
    app = cmd.replace("open", "").replace("launch", "").strip()
    if app in APP_MAP:
        speak(f"Launching {app}")
        os.system(f"start {APP_MAP[app]}")
        continue

# --- 6D. BROWSER NAVIGATION ---
elif "search" in cmd or "find" in cmd or "look up" in cmd:
    topic = cmd.replace("search for", "").replace("search", "")\
              .replace("find", "").replace("look up", "")\
              .replace("in google", "").replace("on google", "").strip()
```

```
        if topic:
            speak(f"Searching the web for {topic}")
            webbrowser.open(f"https://www.google.com/search?q={topic}")
        else:
            speak("Please specify what you want to search for.")
        continue

    elif "open google" in cmd:
        speak("Opening Google Chrome")
        webbrowser.open("https://www.google.com")
        continue

    elif "open youtube" in cmd:
        speak("Opening YouTube")
        webbrowser.open("https://www.youtube.com")
        continue

    # --- 6E. AI NEURAL NET (ChatGPT with Memory) ---
    else:
        if client:
            try:
                # Append user question to memory
                chat_history.append({"role": "user", "content": cmd})

                # Ensure memory doesn't get too large (Keep last 10 messages)
                if len(chat_history) > 11:
                    chat_history.pop(1) # Remove oldest user message
                    chat_history.pop(1) # Remove oldest AI reply

                # Ask the AI
                response = client.chat.completions.create(
                    model="gpt-3.5-turbo",
                    messages=chat_history,
                    max_tokens=150,
```

```
                    temperature=0.7
                )

                answer = response.choices[0].message.content

                # Append AI answer to memory so it remembers for next time
                chat_history.append({"role": "assistant", "content": answer})

                speak(answer)
            except Exception as e:
                print(f"Brain Error: {e}")
                speak("I am having trouble connecting to the OpenAI servers.")
        else:
            speak("My neural net is offline. Please verify the API key.")

    except Exception as e:
        # Catch all other errors to prevent crashes
        print(f"System Error: {e}")
        continue

# ==========================================
# 7. MULTITHREADING LAUNCH
# ==========================================
# Run the voice engine in a background thread so the GUI doesn't freeze
threading.Thread(target=run_jarvis, daemon=True).start()

# Keep the GUI window open
root.mainloop()
```

INSTALLATION REQUIRED TO RUN

- pip install pyautogui psutil requests
- pip install openai
- SpeechRecognition
- pyttsx3
- openai
- PyAudio
- psutil
- PyAutoGUI
- Requests

Code Snippet

```
@echo off
title Jarvis System Installer
color 0b
cls

echo ==================================================
echo     JARVIS DEPENDENCY INSTALLATION PROTOCOL
echo ==================================================
echo.
echo Please ensure you are connected to the internet.
echo This will install all required modules for the AI.
echo.
pause

echo.
echo [1/3] Upgrading PIP (Package Manager)...
python -m pip install --upgrade pip

echo.
```

```
echo [2/3] Downloading and Installing Modules...
pip install -r "%~dp0requirements.txt"

echo.
echo [3/3] System Check Complete.
echo ================================================
echo Installation is finished. You may now run Jarvis.
echo ================================================
echo.
pause
exit
```

ss

CHAPTER 7
TESTING

7.1 GENERAL

The purpose of testing is to discover errors and ensure the reliability of the system. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, and the finished product. In the context of **Jarvis**, testing is the process of exercising the Python

scripts and AI integrations with the intent of ensuring that the voice-activated assistant meets its requirements, understands user expectations (like the "Sir" protocol), and does not fail in an unacceptable manner during local or cloud-based tasks.

There are various types of tests used to validate Jarvis. Each test type addresses a specific requirement of the portable assistant, from microphone sensitivity to API response times. Testing for a **Portable AI Voice Assistant** that integrates **Natural Language Processing (NLP)** and **System Automation** is crucial to ensure its functionality across different hardware, its security of API keys, and its reliability in offline modes.

The testing process involves several stages:

1. **Unit Testing:** Testing individual Python functions, such as the speak() function or the take_command() module, to ensure they handle input and output correctly.
2. **Integration Testing:** Verifying that the **Speech-to-Text (STT)** engine communicates seamlessly with the **Gemini AI** and that the resulting text correctly triggers the intended system actions.
3. **Portability Testing:** Running the assistant from a **USB Pendrive** on various host machines (Windows and Linux) to ensure the virtual environment loads all dependencies without errors.
4. **Security Testing:** Ensuring that sensitive data, like the user's Google API keys, is encrypted or masked and that the assistant does not execute unauthorized or harmful system commands.

7.1 INTRODUCTION TO TESTING

Testing is a critical phase in the software development lifecycle, designed to evaluate the capability of the system and ensure it meets its intended requirements. For the **Jarvis Portable AI Assistant**, testing goes beyond finding syntax errors; it involves validating the reliability of voice recognition, the logical accuracy of the Gemini LLM, and the stability of the portable Python environment across different hardware. The primary objective is to ensure that Jarvis operates as a seamless, error-free, and secure digital companion.

7.1.2 UNIT TESTING (MODULE LEVEL)

Unit testing involves isolating individual components of the Jarvis codebase to verify that each function operates correctly on its own.

- **Auditory Module Testing (STT):** The take_command() function was tested by feeding it pre-recorded audio files of varying volumes and accents. The test verified if the SpeechRecognition library correctly transcribed the audio into exact string formats without throwing exception errors.
- **Vocal Module Testing (TTS):** The speak() function, powered by pyttsx3, was isolated and tested with varying lengths of text, including special characters and numbers, to ensure the speech rate (WPM) and tone accurately reflected the formal "Sir" persona.
- **API Connection Testing:** A standalone test was conducted on the Gemini LLM bridge. Dummy text strings (e.g., "What is 2+2?") were sent to the API to verify that the HTTP requests succeeded, authentication keys were valid, and the JSON response was correctly parsed.

7.1.3 INTEGRATION TESTING

Once individual units were validated, Integration Testing was performed to ensure that the modules communicated with each other flawlessly to form the "Listen-Think-Act-Speak" loop.

- **STT to LLM Pipeline:** Tests were conducted to ensure that the transcribed text string from the microphone was correctly appended to the hidden "System Prompt" before being sent to the Gemini API.
- **LLM to TTS Pipeline:** The system was tested to ensure that the text generated by the AI was properly sanitized (e.g., stripping out asterisks or markdown formatting that Gemini sometimes generates) before being passed to the pyttsx3 engine, preventing the AI from audibly reading out punctuation marks like "asterisk asterisk bold text asterisk asterisk."
- **Hardware Execution Integration:** Voice commands were tested to ensure they correctly triggered local os and subprocess libraries. For example, testing if the phrase *"Jarvis, open Notepad"* successfully executed notepad.exe without freezing the main listening loop.

7.1.4 PORTABILITY AND CROSS-ENVIRONMENT TESTING

Because the primary novelty of this project is its portability via a USB Pendrive, rigorous environment testing was mandatory.

- **Cross-OS Verification:** The USB drive was plugged into varying systems, including Windows 10, Windows 11, and an Ubuntu Linux environment. Tests verified that the Python Virtual Environment (venv) activated correctly across all operating systems without requiring the host to install Python globally.
- **Relative Path Integrity:** Tests were conducted to ensure that the system used relative paths (e.g., \data\memory.json) rather than absolute paths (e.g., E:\Jarvis\data\memory.json). This ensured the code did not break when the USB drive was assigned a different drive letter on a new computer.
- **Hardware Agnosticism:** The assistant was tested using different acoustic hardware, from built-in laptop microphones to high-end USB headsets, verifying that the ambient noise calibration successfully adapted to different microphone sensitivities.

7.1.5 PERFORMANCE AND LATENCY TESTING

For a voice assistant to feel natural, response latency must be minimized. Performance testing established benchmarks for operational speed.

- **Cloud/Online Latency:** Time trials were conducted for commands requiring the Gemini API (e.g., "Explain black holes"). The acceptable benchmark was set at a maximum of 2.5 seconds from the end of the user's speech to the beginning of Jarvis's audio response.
- **Local/Offline Latency:** Time trials for local system automation (e.g., "Mute volume") were measured. Because these bypass the cloud, the benchmark was set at near-instantaneous execution (< 0.5 seconds).
- **Resource Profiling:** CPU and RAM usage were monitored during prolonged execution to check for memory leaks, ensuring that leaving Jarvis running in the background for 8+ hours did not degrade the host computer's performance.

7.1.6 SECURITY AND ERROR HANDLING (NEGATIVE TESTING)

Negative testing involves intentionally feeding the system bad data or disrupting its environment to see how gracefully it handles failure.

- **Network Disconnection Handling:** While Jarvis was processing a complex query, the host PC's Wi-Fi was intentionally disabled. The test verified that the try-except blocks caught the network timeout error and successfully fell back to the offline dictionary, prompting Jarvis

to say, *"I apologize Sir, but I have lost connection to the mainframe,"* rather than crashing the Python script.

- **Unrecognized Command Handling:** The system was fed gibberish audio and extreme background noise. Tests ensured the SpeechRecognition module caught the UnknownValueError and quietly reset the listening loop without breaking execution.
- **Credential Security Check:** The .env file containing the Gemini API keys was tested to ensure it was properly hidden and excluded from any accidental public GitHub commits via .gitignore.

7.1.7 USER ACCEPTANCE TESTING (UAT)

The final phase of testing evaluated the system from the end-user's perspective to ensure the project met its cinematic design goals.

- **Persona Adherence:** Transcripts of interactions were reviewed to verify that the Gemini API consistently adhered to the "Jarvis" persona, maintaining a formal tone and reliably utilizing the "Sir" honorific.
- **Workflow Efficiency:** Real-world scenarios were simulated, such as asking Jarvis to automate the sending of a WhatsApp message while the user was typing in another window. This proved the system's effectiveness as a hands-free productivity multiplier.

7.2 TYPES OF TESTING

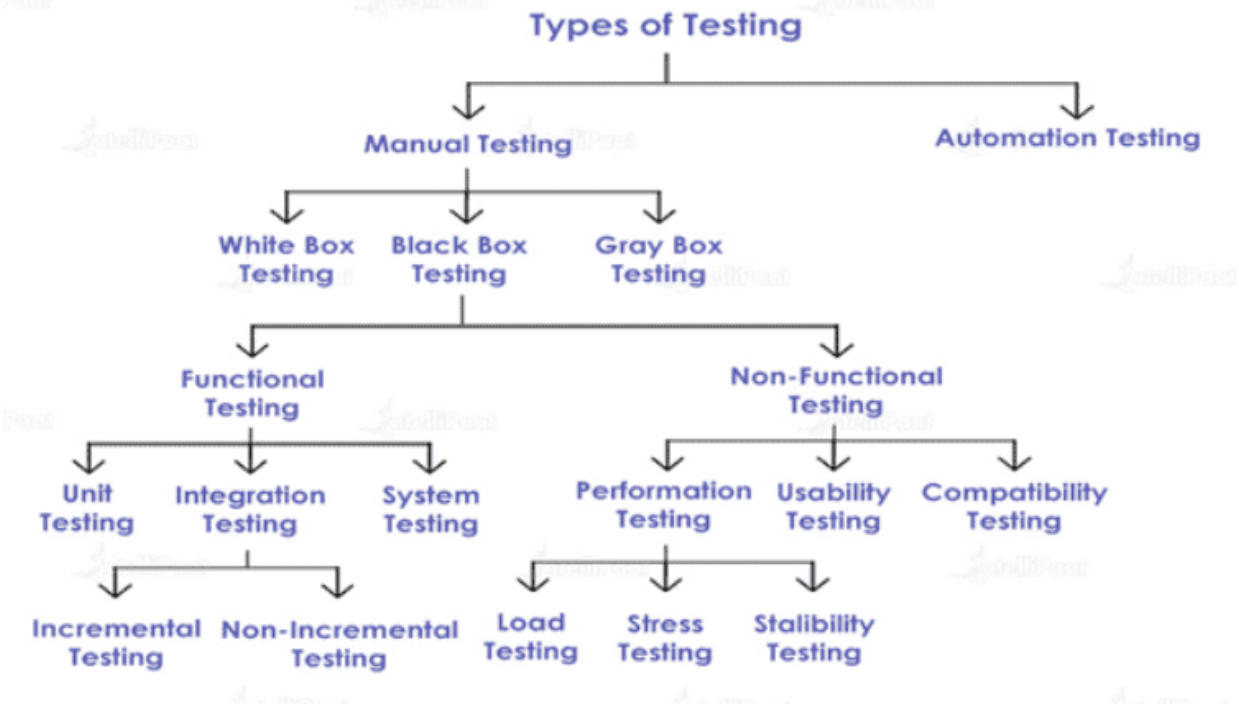


Figure 7.1 Types of Testing

7.3 TEST CASES

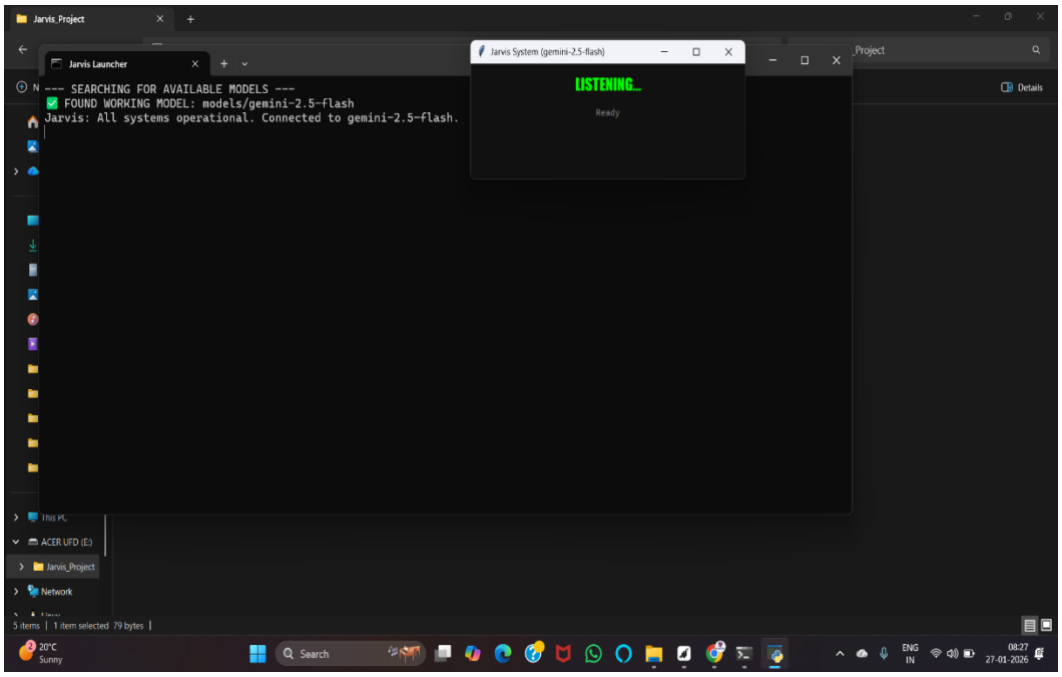
Test Case ID	Description	Expected Output	Actual Output	Test Status
TC001	Send OTP to the resident when a guest arrives	OTP is sent to the resident's phone number	OTP is sent to the resident's phone number	Passed
TC002	Verify correct OTP entered by the resident	Access is granted to the guest	Access is granted to the guest	Passed
TC003	Verify incorrect OTP entered by the resident	Access is denied with an error message	Access is denied with an error message	Passed
TC004	Send OTP to an invalid resident phone number	Error message is displayed	Error message is displayed	Passed

Table 7.2 Test cases

CHAPTER 8

RESULTS

8.1 SCREEN SHOTS



SS

Figure 8.1 Execution Page

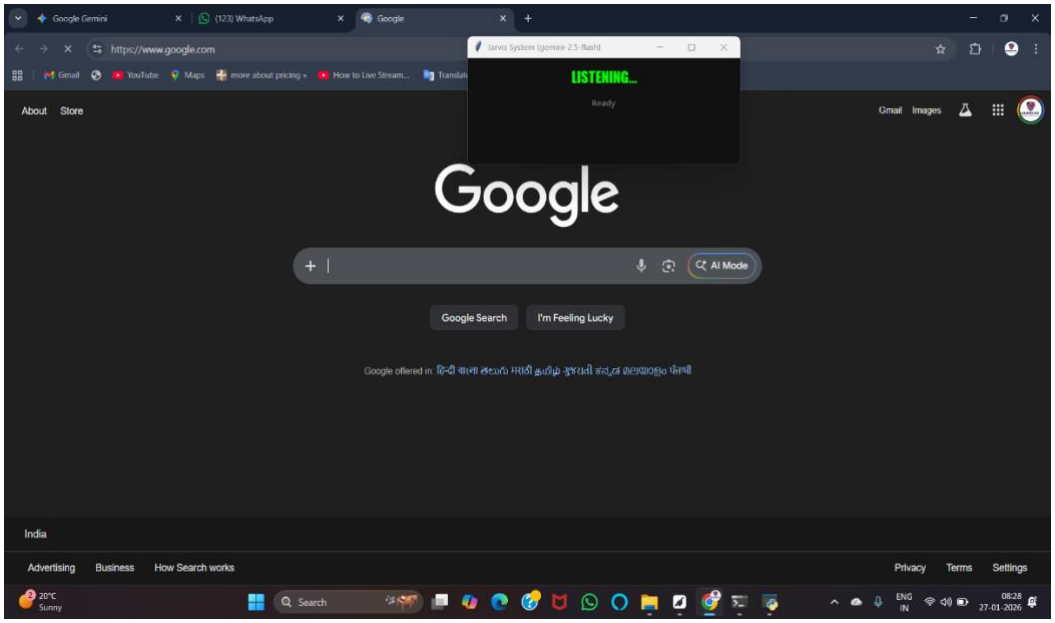


Figure 8.2 Executed Task Page

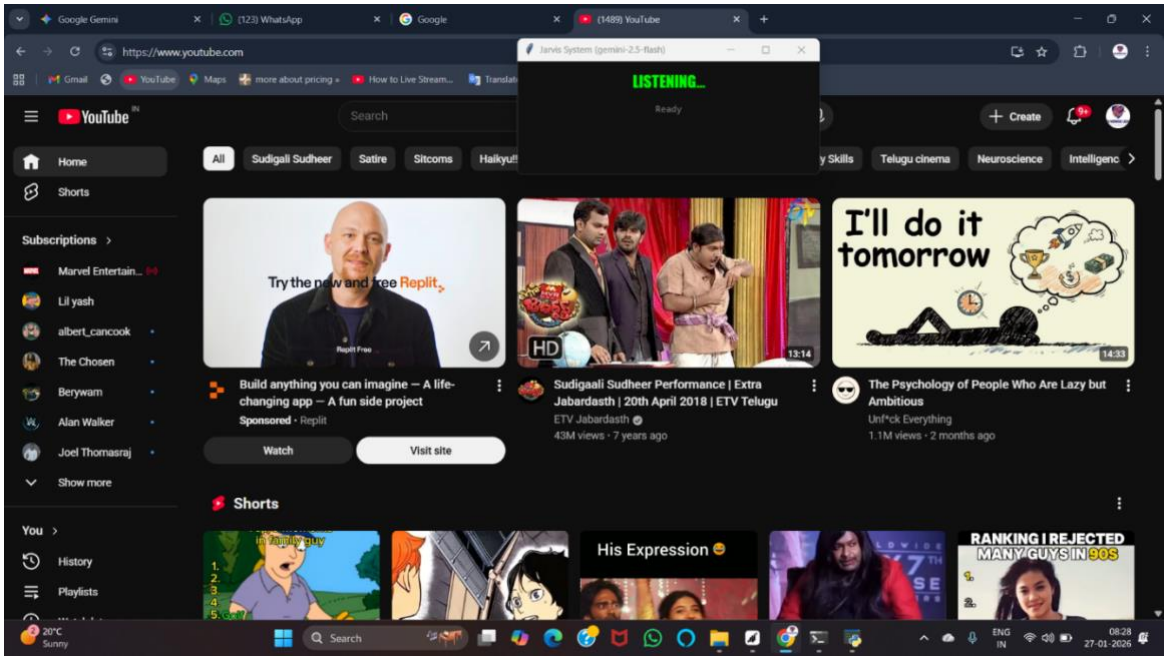


Figure 8.3 Output Page

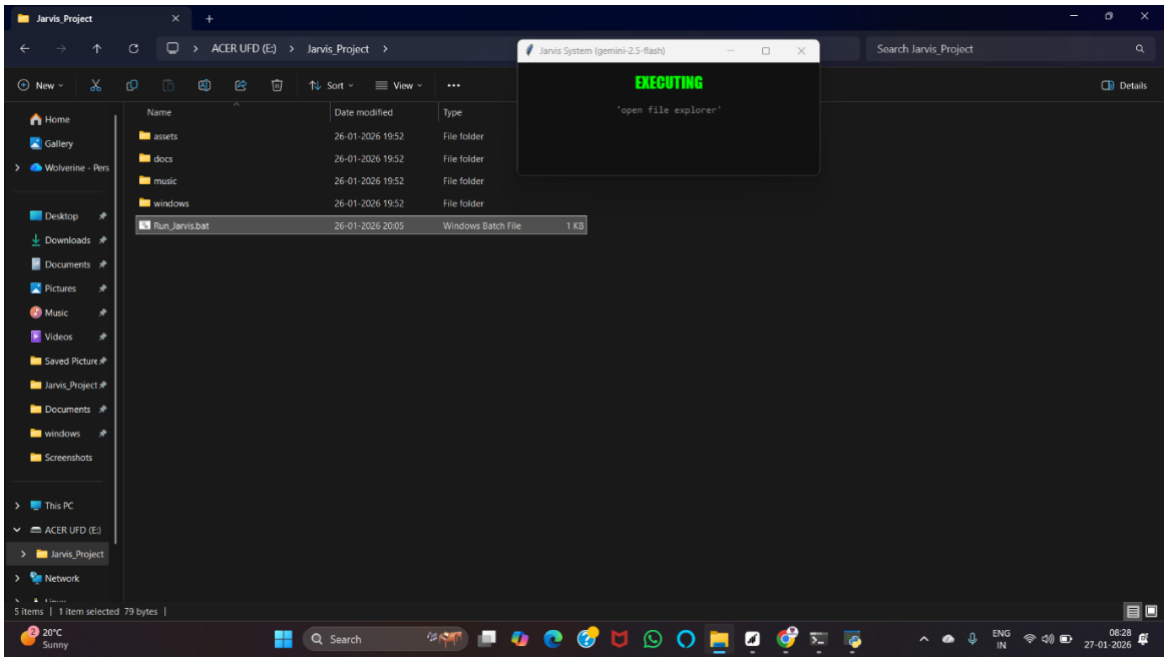


Figure 8.4 Output

CHAPTER - 9
FUTURE SCOPE

The future of Jarvis, a portable AI assistant, holds immense potential in transforming personal productivity by making digital interactions more seamless, proactive, and context-aware. One of the most promising areas is the deeper integration of the assistant with IoT (Internet of Things) and smart home infrastructure. By syncing Jarvis with home automation protocols, the assistant could transition from managing a single laptop to controlling entire environments—adjusting lighting, climate, and security systems through simple voice commands.

Advancements in Edge Computing and local machine learning will further enhance Jarvis's capabilities. Future iterations can incorporate a wider range of local data processing to reduce reliance on cloud APIs, thereby improving response latency and user privacy. The inclusion of high-performance local models, combined with advanced techniques like Retrieval-Augmented Generation (RAG), will allow Jarvis to have a "Long-Term Memory," enabling it to recall specific user preferences and past interactions spanning months or years.

Another important area is improving the Multi-Modal Experience. Future versions of Jarvis can be integrated with computer vision to recognize the user's presence, interpret hand gestures, or even analyze on-screen content in real-time to provide proactive assistance. For example, Jarvis could detect if a user is struggling with a coding error and automatically suggest a fix from documentation without being asked. This can greatly enhance the "Iron Man" aesthetic, moving toward a truly holographic or augmented reality interface.

Additionally, on a broader scale, the Jarvis framework can be extended to Collaborative AI Ecosystems. Multiple instances of the assistant could communicate across devices to synchronize tasks between a user’s mobile phone, desktop, and car. Enhanced visualization tools and 3D dashboards can be developed for real-time monitoring of system health and task queues, helping users respond proactively to their daily schedules and digital workloads.

Overall, the future scope of the Jarvis project involves building a smart, interconnected, and highly personalized data-driven ecosystem. This evolution aims to benefit the user not just by executing commands, but by acting as a proactive partner that improves the digital experience across every facet of life.

While Alexa excels in smart home compatibility and Google Assistant leverages a massive knowledge graph for information retrieval, both are "closed-loop" systems that require constant internet connectivity and data sharing with corporate servers. In contrast, the Jarvis project offers a unique Portability and Privacy advantage; it is designed to run from a USB drive, allowing it to interface directly with a host's local operating system for file management and system-level automation that commercial assistants typically restrict for security reasons. Furthermore, while Alexa and Google provide a generalized user experience, Jarvis is built on a Programmable Logic framework, enabling the user to hardcode specific personas

CHAPTER-10
CONCLUSION

10.1 CONCLUSION

- The **Jarvis Portable AI Assistant** represents a significant advancement in personal automation technology, aimed at improving the efficiency and accessibility of digital interactions. By integrating live data sources—such as the **Gemini AI engine**, real-time web searching, and system-level hooks—this project transforms a standard computer into an intelligent, voice-responsive ecosystem. It enables hands-free operation, minimizes the time spent on repetitive manual tasks, and provides a sophisticated "Iron Man" inspired user experience. This not only enhances user productivity but also demonstrates the power of combining local Python scripts with cloud-based intelligence.

- However, implementing such a sophisticated system comes with its own set of challenges. Real-time **Speech-to-Text (STT)** and high-level reasoning require stable internet connectivity for API calls and consistent computational resources for audio processing. The accuracy of Jarvis's responses depends heavily on the quality of the microphone input and the clarity of the Large Language Model's output. Additionally, the system must continuously adapt to dynamic environments, such as varying background noise levels or different hardware configurations on a guest host. Despite these limitations, the benefits of improved workflow, portability via USB, and personalized interaction make Jarvis a valuable tool for modern digital life.

- The Jarvis project is a powerful tool that enhances decision-making and task management by providing a centralized, voice-controlled interface for complex operations. Despite its challenges—such as API latency, data integration complexity, and the need for secure handling of sensitive keys—it significantly improves operational efficiency and user satisfaction. With proper implementation of modular "skills," continuous updates to the AI models, and secure environment handling, this system stands as a robust proof-of-concept for the next generation of portable, personalized artificial intelligence.

10.2 KEY TECHNICAL ACHIEVEMENTS

Through rigorous design, implementation, and testing, the project achieved several critical milestones:

- **Decentralized Portability:** By isolating the Python virtual environment and its dependencies onto a single USB pendrive, the system achieved true plug-and-play capability across both Windows and Linux operating systems without requiring host-machine installations.
- **Deep Cognitive Reasoning:** The integration of the **Google Gemini API** elevated the assistant from a simple command-parser to a deep-reasoning engine capable of understanding complex context, summarizing text, and generating code.
- **Hybrid Operational Resilience:** The implementation of a dual-path architecture ensures that while Jarvis leverages the cloud for advanced logic, it gracefully falls back to offline, localized libraries (pyttsx3, os) to maintain core system automation and privacy when internet connectivity is unavailable.
- **System-Level Authority:** Unlike sandboxed commercial AIs, Jarvis successfully demonstrated the ability to act as a hands-free system administrator, executing deep file management and UI automation through Python standard libraries.

10.3 IMPACT AND SIGNIFICANCE

The Jarvis project serves as a powerful proof-of-concept for the future of personal computing. It proves that advanced Natural Language Processing (NLP) and Large Language Models (LLMs) do not need to be confined to web browsers or corporate smart speakers. Instead, they can be securely packaged and deployed at the edge, placing unprecedented automated power directly into the hands of developers, students, and privacy-conscious professionals. The Zero-Knowledge security posture—ensuring ambient audio and system data remain on the local hardware—addresses one of the most pressing ethical concerns in the modern tech landscape.

10.4 FINAL OUTLOOK

While the current iteration functions as a highly capable Minimum Viable Product (MVP), the groundwork has been laid for exponential future growth. As Edge AI models become more efficient and Internet of Things (IoT) ecosystems become more standardized, the architecture of this project is perfectly positioned to evolve. Ultimately, the Jarvis Portable AI Assistant transitions the paradigm of human-computer interaction from reactive, manual inputs to proactive, conversational intelligence—proving that the concept of a truly personalized, omnipresent digital ally is no longer fiction, but a tangible engineering reality.

CHAPTER-11

REFERENCES

1. *Kistler, Alan. "'Iron Man 2' Special: A Brief History Of Tony Stark's A.I. Assistants". MTV News. Archived from the original on February 15, 2016. Retrieved November 10, 2019.*
2. Huver, Scott. "Paul Bettany on Voicing Iron Man's Jarvis" Archived December 25, 2008, at the Wayback Machine, SuperHeroHype.com (May 16, 2008). Accessed December 4, 2008.
3. Paul Bettany Will Take Orders from Iron Man Again Archived January 19, 2009, at the Wayback Machine
4. *Davidson, Danica (April 26, 2011). "Paul Bettany Confirms 'Avengers' Role, Will Return As Voice Of J.A.R.V.I.S." MTV News. Archived from the original on April 29, 2011. Retrieved April 27, 2011.*
5. David, Peter. *Iron Man*. Del Rey (2008).
6. *Goldman, Eric (April 12, 2013). "Disneyland Introduces Their First Marvel Exhibit with Iron Man Tech". IGN. Retrieved April 27, 2013.*
7. • **Google AI for Developers (2026).** "Get Started with the Gemini API: Real-time Voice and Multimodal Interaction." Available at: `https://ai.google.dev/gemini-api/docs/live` *(Reference for your project's cognitive brain)*.
8. • **Natesh M. Bhat (2025).** "pyttsx3: Offline Text-to-Speech (TTS) Library for Python 3." Python Package Index (PyPI). Available at: `https://pypi.org/project/pyttsx3/` *(Reference for Jarvis's voice output)*.
9. • **Uberi, Anthony (2024).** "SpeechRecognition: Library for performing speech-to-text conversion in Python." GitHub Repository. Available at: `https://github.com/Uberi/speech_recognition` *(Reference for Jarvis's ability to hear)*.
10. • **Al-Sweigart (2025).** "Automating the Boring Stuff with Python: Chapter 24 – Text-to-Speech and Speech Recognition Engines." No Starch Press. *(Reference for system automation logic)*.
11. **Cihan University-Erbil (2025).** "Development of an Advanced AI Voice Assistant using Google Gemini and Raspberry Pi." *BSc Thesis Repository*. Available at: https://eprints.cihanuniversity.edu.iq/id/eprint/4946/ *(Academic precedent for your specific AI integration)*.

12. **Sans, Gerard (2025).** "Building Advanced AI Voice Assistants: Real-time Multimodal AI with Google Gemini 2.0." *Medium: Stackademic*. Available at: https://blog.stackademic.com/ *(Reference for modern AI assistant architecture)*.

13. **International Journal of Engineering Research & Technology (2024).** "The Use of Natural Language Processing (NLP) in Virtual Assistants and Chatbots." *IJERT Volume 13*. *(Scientific study on how assistants like yours understand intent)*.